Laboratorio 3 - Redes y sistemas distribuidos

ALGORITMOS DE CONTROL DE FLUJO Y CONGESTIÓN

Integrantes:

- Arri Hartwig, Camila
- Borioni, Valentín
- Gatti, Julián
- Piccotto, Marcos

Profesor a cargo: Bordone, Matías

Fecha: 13 de mayo de 2025

Introducción

En este laboratorio se realiza un análisis del comportamiento de una red de comunicaciones utilizando OMNeT++, un entorno de simulación modular y extensible diseñado para el estudio de redes de computadoras y sistemas distribuidos.

El objetivo principal de este informe es evaluar el rendimiento de una red bajo determinadas condiciones de tráfico y configuración, presentando especial atención a métricas clave como el tamaño de los buffers, los retardos de transmisión y el comportamiento dinámico del sistema frente a la carga generada. Para ello, se diseñó un modelo de red simple compuesto por nodos transmisores, colas de enrutamiento y nodos receptores, en el cual se simulan eventos de envío, recepción y acumulación de paquetes durante un intervalo de tiempo definido.

A través del análisis de los datos obtenidos, se realizaron estadísticas relevantes que permiten interpretar el rendimiento de la red en función de los parámetros configurados.

OMNet++

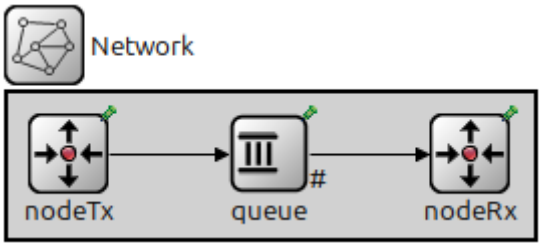
Las características técnicas y pedagógicas que presenta OMNet++ lo convierten en una plataforma ideal para el análisis y modelado de redes de comunicación.

Una de sus principales ventajas es su enfoque orientado a eventos discretos, lo cual lo hace especialmente adecuado para representar el comportamiento temporal de las redes, donde las acciones ocurren de manera secuencial y dependiente del tiempo. Esto permite simular con precisión fenómenos como la acumulación de tráfico, la saturación de buffers, y los retrasos en la entrega de paquetes, que son aspectos clave del presente análisis.

Además proporciona herramientas integradas para la recolección, visualización y análisis de resultados, como los archivos `.vec` y `.sca`, que facilitan la obtención de métricas precisas y reproducibles.

Modelo de red

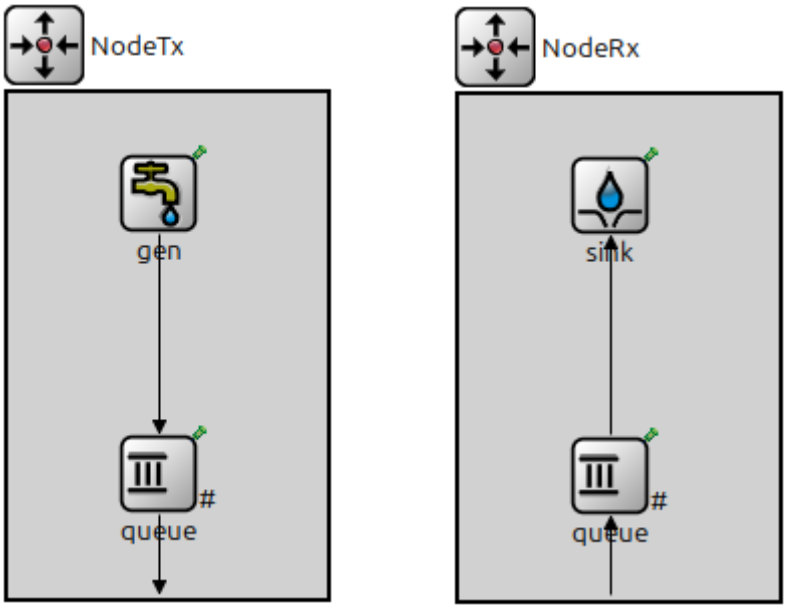
El modelo de red utilizado contiene un nodo emisor (`NodeTx`), un nodo receptor (`NodeRx`) y una queue intermedia:



Cada nodo tiene la siguiente funcionalidad:

- `NodeTx:` generador que envía los paquetes a una queue.
- `NodeRx:` queue donde se almacenan los paquetes recibidos y los envía al `Sink`.

Y su estructura interna es:



Parámetros para la simulación

```
Network.nodeTx.gen.generationInterval=exponential(0.1)
Network.nodeTx.gen.packetByteSize = 12500
Network.nodeTx.queue.bufferSize = 2000000
Network.queue.bufferSize = 200
Network.nodeRx.queue.bufferSize = 200
```

- `generationInterval:` frecuencia con la que el nodo genera los paquetes (distribución exponencial centrada en 0.1).
- `packetByteSize:` tamaño de los paquetes (12500 Bytes).
- `bufferSize:` tamaño de los diferentes buffers:
  - `NodeTx`.
  - `queue` (intermedia entre los nodos)
  - `NodeRx.`

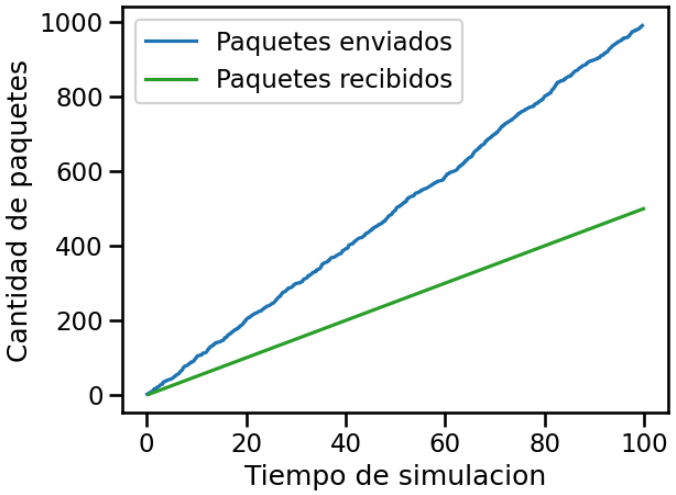
Para comparar diferentes comportamientos en nuestra red realizamos experimentos cambiando la frecuencia con la que se envían los archivos sobre dos casos específicos:

- **Caso de estudio 1**
  `NodeTx` a `Queue`: datarate = 1 Mbps y delay = 100 us
  `Queue` a `NodeRx`: datarate = 1 Mbps y delay = 100 us
  `Queue` a `Sink`: datarate = 0.5 Mbps

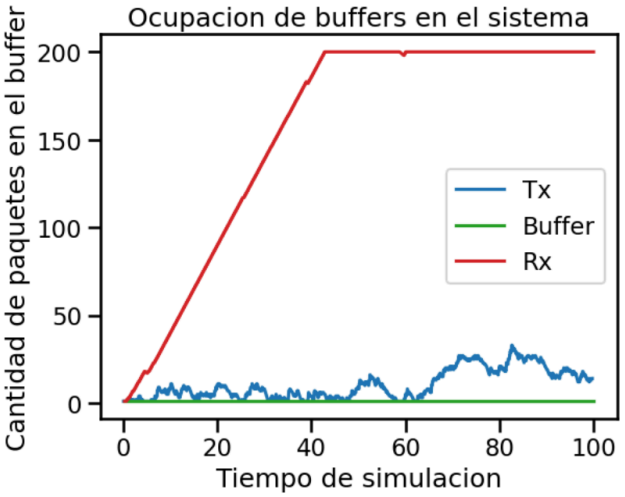
- **Caso de estudio 2**
  `NodeTx` a `Queue`: datarate = 1 Mbps y delay = 100 us
  `Queue` a `NodeRx`: datarate = 0.5 Mbps y delay = 100 us
  `Queue` a `Sink`: datarate = 1 Mbps

Estudio del caso 1

Al analizar la cantidad de paquetes enviados frente a los recibidos se observa que muchos no llegan al receptor
.



Debido a la ocupación de los buffer se identifica que el problema está en `NodeRx`, cuyo buffer se llena y descarta paquetes.

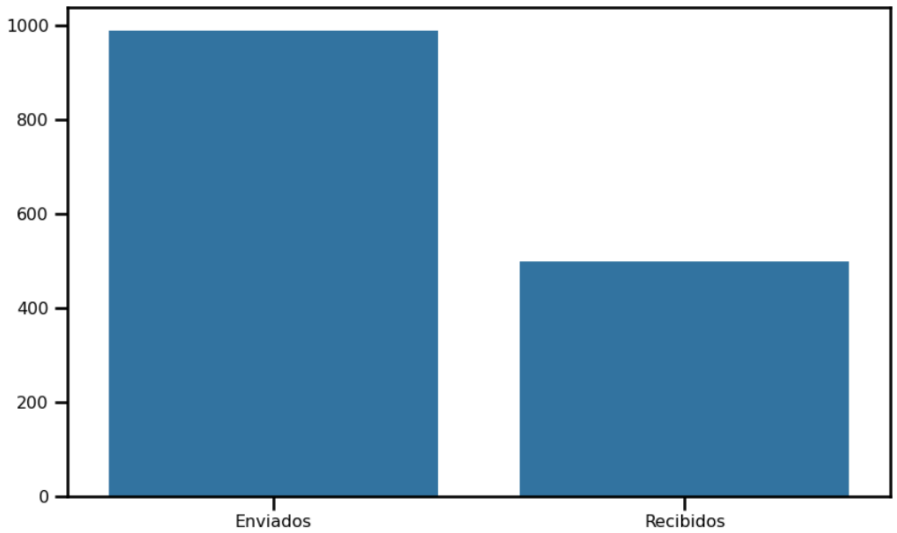


Claramente este es un problema de flujo ya que los paquetes están llegando al nodo receptor pero, al estar lleno, los está descartando.

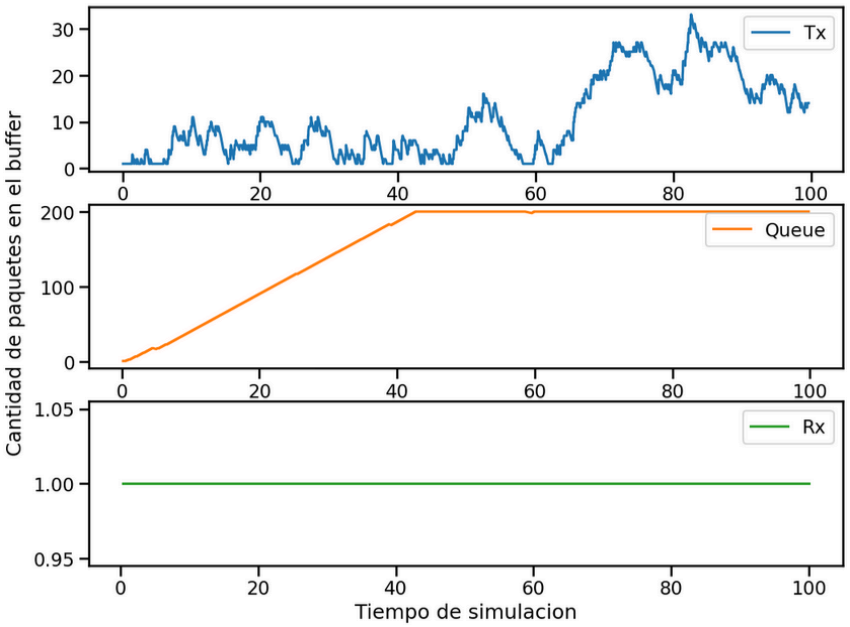
Este problema de flujo se debe a un cuello de botella: el buffer del nodo receptor recibe datos a 1 Mbps pero sólo puede enviarlos a 0.5 Mbps hacia el `Sink`, causando colapso del buffer y pérdida de paquetes.

Estudio del caso 2

En la simulación del caso 2 sucede algo parecido ya que también se pierden paquetes, pero al observar los buffers se identifica que en este caso el buffer que se satura es el de la Queue lo que resulta en un problema de congestión.



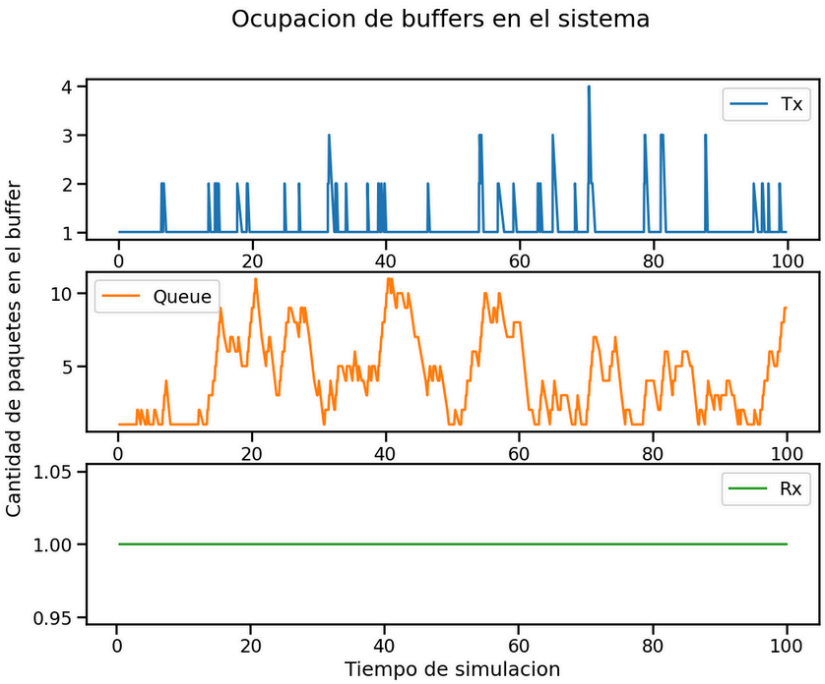
Ocupacion de buffers en el sistema



Esto es esperable ya que el datarate de la Queue al nodo receptor es de 0.5Mbps lo que genera un cuello de botella pues llegan paquetes más rápido de lo que se envían.

Ejecutando diferentes simulaciones y cambiando los parámetros se observa que atrasar el tiempo en el que se crean los paquetes (`generationInterval`) provoca que los buffers no se colapsen, independientemente del caso que estemos viendo, lo cual tiene sentido ya que si los paquetes se generan lo suficientemente espaciados uno de otro, le damos tiempo a los buffer a que se vacíen.

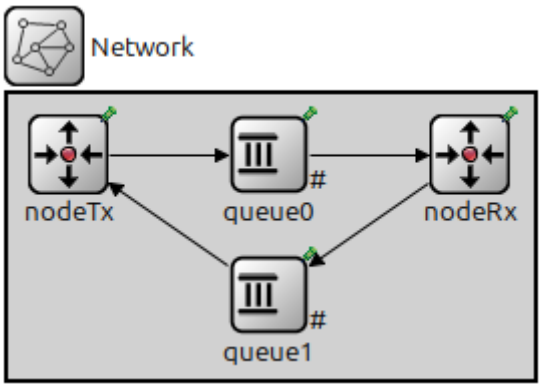
Eso genera la siguiente pregunta: ¿cuál es el tiempo mínimo en el que logramos que no se pierdan paquetes?. La respuesta es: `generetionInterval=exponential(0.2)`, es decir, que los paquetes se crean con una distribución exponencial centrada en 0.2s. Podemos ver la ocupación de los buffers en la siguiente imagen.



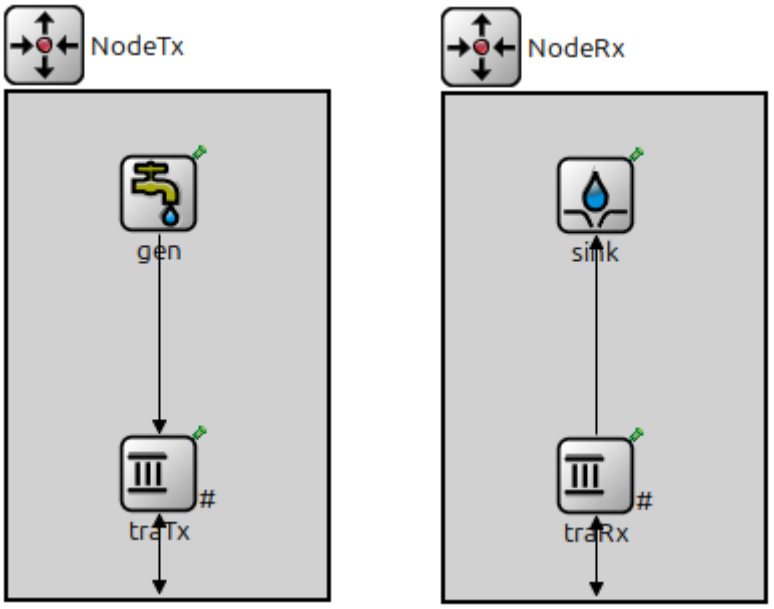
Control de flujo

El control de flujo es una técnica para evitar que el emisor envíe más paquetes de los que el receptor puede procesar. Esto es necesario ya que, al no controlar el flujo, el nodo receptor se podría saturar provocando pérdida de paquetes, como se observó en el estudio del caso 1.

Para poder implementar el control de flujo el nodo receptor debería poder comunicarle al nodo emisor cuan saturado está. Para ello se definió un nuevo modelo de red.



Se puede observar la implementación de una segunda cola (`queue1`) mediante la redefinición de los nodos con una nueva cola interna (`traTx` y `traRx`) que permite tanto enviar como recibir mensajes. Esta segunda cola se utiliza para la comunicación entre el nodo receptor con el emisor.

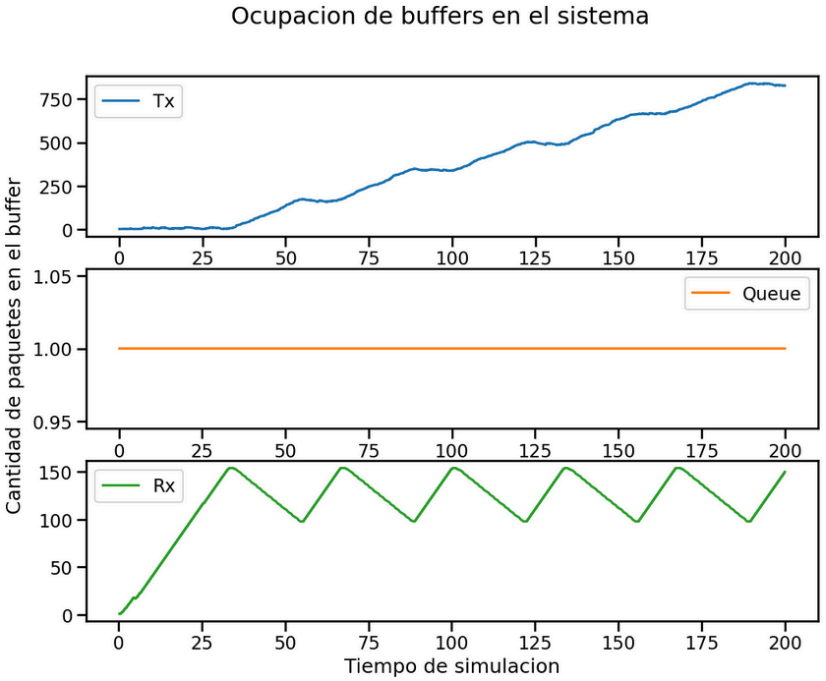


Entonces el funcionamiento sería: nodeRx recibe el paquete y lo envía al sink, pero además, arma un mensaje que contiene información sobre el estado actual de su capacidad y envía este mensaje mediante `queue1`, `NodeTx` recibe esta información y la procesa tomando la decisión de retrasar, acelerar o no modificar el envío de los paquetes; dependiendo de si el buffer del receptor está saturado, tiene mucho espacio libre o se encuentra en un punto óptimo respectivamente. La saturación se considera de la siguiente manera:

- Si la ocupación del buffer es mayor al 50% total, se ralentiza el envío de paquetes en 0.2.
- Si la ocupación del buffer es menor al 25% total, se acelera el envío de paquetes en 0.1.
- Si la ocupación del buffer se encuentra entre el 20% y el 50% del tamaño total, se considera una ocupación óptima, i.e, la velocidad de envío se mantiene.

```
void TransportTx::flowControl(FeedbackPkt *msg)
{
   // Obtenemos el tamaño total del buffer
   int bufferSize = msg->getBufferSize();
   // Obtenemos la ocupación actual del buffer
   int currentBufferSize = msg->getCurrentBufferSize();
   // Calculamos el espacio libre del buffer
   double ratio = (double)(bufferSize - currentBufferSize) / bufferSize;
   // Si queda libre menos del 25% del espacio del buffer
   if (ratio <= 0.25)
   {
       delay = 0.1;
   }
   // Si queda libre más del 50% del espacio del buffer
   else if (ratio > 0.5)
   {
       delay = -0.2;
   }
   // Si se encuentra en un punto óptimo
   else
   {
       delay = 0;
   }
}
```

Este comportamiento podemos observar con la siguiente gráfica:



La figura muestra la evolución temporal de la ocupación de buffers en el sistema tras implementar el algoritmo de control de flujo.

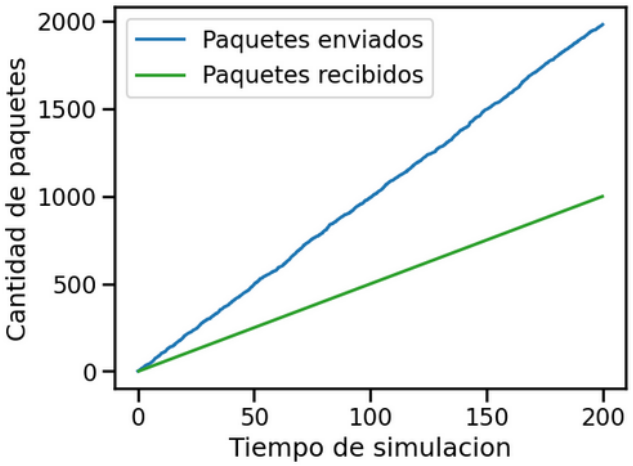
En el primer cuadro se observa que el emisor continúa generando paquetes de manera sostenida a lo largo del tiempo, sin embargo, la pendiente de crecimiento indica que el ritmo de transmisión se adapta gradualmente, evitando un crecimiento abrupto lo que indicaría que la transmisión se realizará de forma muy lenta. Asimismo, en el tercer cuadro que corresponde al nodo receptor es cíclico, con una pendiente de crecimiento seguida de un descanso. A pesar de estas oscilaciones, la recepción no supera un umbral crítico, lo que garantiza un procesamiento estable y continuo.

Con los diferentes experimentos se notó que si bien ya no se pierden paquetes, el algoritmo presenta una ineficiencia importante: los paquetes se acumulan progresivamente en el emisor.

Esto se observa en la primera gráfica, donde la cantidad de paquetes en el buffer del transmisor crece de forma sostenida a lo largo del tiempo, sin señales de estabilización o drenaje.

La acumulación indica que el emisor está reteniendo paquetes en espera de poder enviarlos, lo cual implica los siguientes problemas:

- Baja eficiencia temporal: los datos generados no se transmiten inmediatamente, lo que aumenta la latencia y reduce el rendimiento.
- Sobrecarga de almacenamiento en el emisor: si el programa continúa con el tiempo, eventualmente se podría llenar el buffer lo que ocasionará pérdidas de paquetes.



Control de congestión

La congestión ocurre cuando la demanda de recursos de red excede su capacidad de procesamiento o transmisión, lo que provoca una acumulación excesiva de paquetes en los nodos intermedios.

Esta situación genera una serie de consecuencias negativas como pérdidas de paquetes, incremento en la latencia, retransmisiones innecesarias entre otras.

¿Cómo identificar que hay congestión en la red?

Mediante la implementación de un algoritmo que mide el tiempo que viaja un paquete en la red, desde la espera y salida de `NodeTx` hasta la llegada del mensaje a `NodeRx`, y calculando un promedio de este tiempo sobre todos los paquetes.

Entonces conforme llegan los paquetes, el emisor recibe la información del tiempo que le tomó a un paquete viajar y verifica si está por encima, por debajo de un umbral, para saber si acelerar, retrasar o no modificar la velocidad con la que se envían los paquetes respectivamente. El umbral está definido de la siguiente manera:

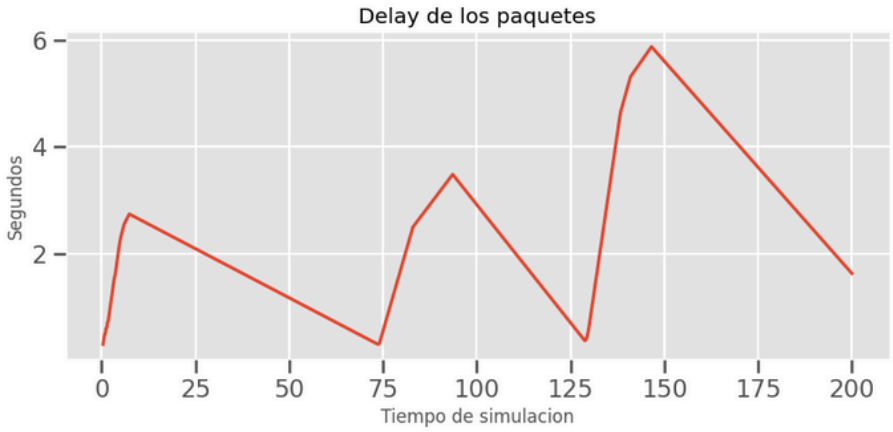
- Si el paquete actual demora más de 1.2 veces el promedio de viaje calculado, se debe ralentizar en 0.2 el envío de paquetes. En este caso también se resetea el promedio para tener la información actualizada de esta demora.
- Si el paquete actual demora menos de 0.2 veces el promedio de viaje calculado, se debe acelerar en 0.1 el envío.
- Si se encuentra entre 1.2 y 0.2 veces el promedio de viaje calculado, se mantiene la velocidad actual. Es decir, este rango de velocidad de viaje es considerado como el ideal para que no haya congestión.

Aclaración de variables:

- msgTimStamp = tiempo de viaje del paquete.
- timeStampMean = promedio de tiempo de viaje de los paquetes.
- countTimeStamp = #(msgTimStamp).
- isFirst = booleano inicialmente declarado como false. Utilizado para que únicamente los primeros 10 paquetes no modifiquen el delay, sino que con ellos se calcula el promedio.
- delay = valor por el que se escala la velocidad de envío de paquetes.

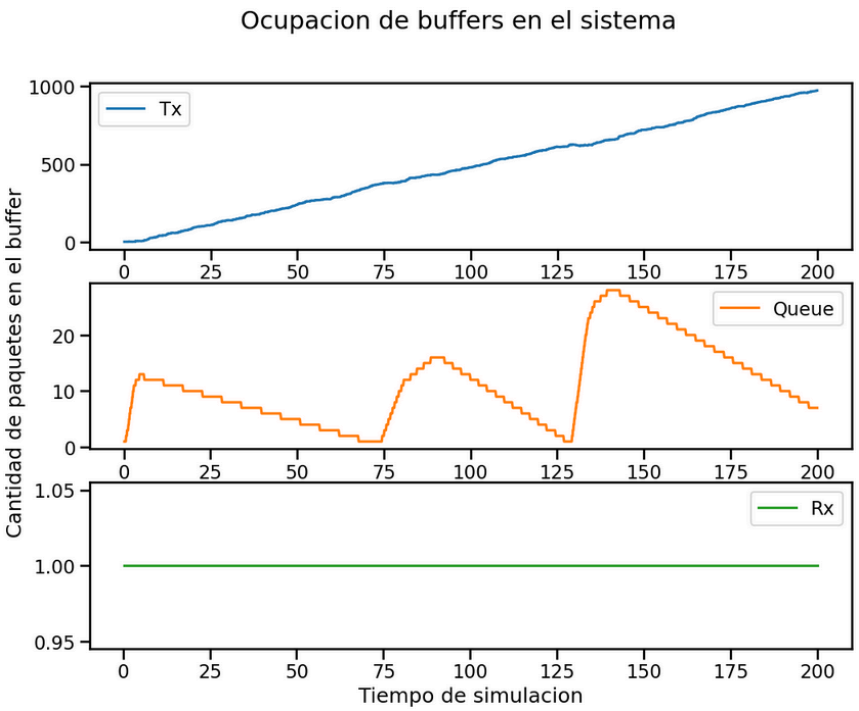
```
void TransportTx::congestionControl(FeedbackPkt *msg)
{
   // Obtenemos el tiempo que tardó el mensaje
   simtime_t msgTimeStamp = msg->getTimeStampRx();
   // Se inicializa el delay en 0, i.e, sin modificaciones
   delay = 0;

   // Luego de la llegada de los 10 primeros paquetes únicamente
   // al iniciar la ejecución, para ello se utiliza la flag isFirst.
   if (countTimeStamp > 10 || isFirst)
   {
       // Umbral para ralentizar el envío.
       if (msgTimeStamp > timeStampMean * 1.2)
       {
           delay = 0.2;
           timeStampMean = msgTimeStamp;
           countTimeStamp = 1;
       }
       // Umbral para acelerar el envío.
       else if (msgTimeStamp < timeStampMean * 0.2)
       {
           delay = -0.1;
       }
       isFirst = true;
   }
    // Actualizamos el promedio teniendo en cuenta el tiempo de viaje
del
   // paquete   actual
    timeStampMean = ((timeStampMean * countTimeStamp) + msgTimeStamp) /
(countTimeStamp + 1);
   countTimeStamp++;
}
```

Este comportamiento lo podemos observar en la siguiente figura, donde la evolución del delay que experimentan los paquetes sigue un patrón muy similar al de la ocupación del buffer intermedio.

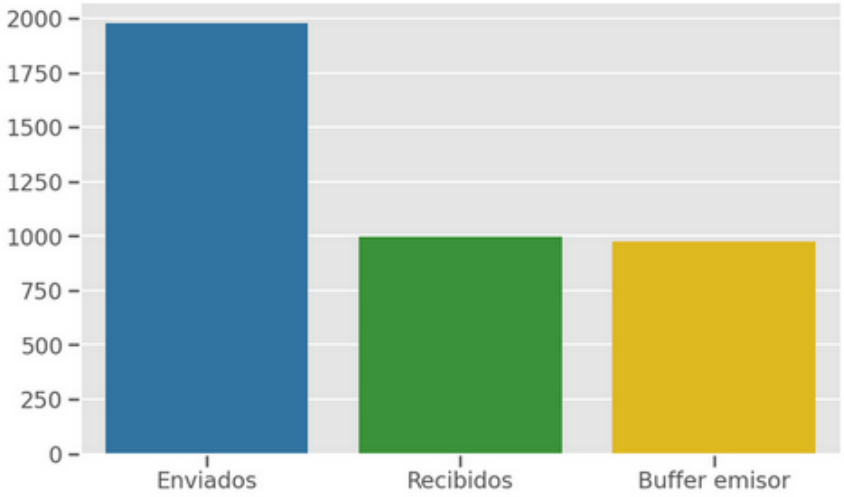


Esto no es una coincidencia. Cuando el buffer se encuentra más ocupado, los paquetes nuevos que llegan deben esperar más tiempo para ser procesados. Este tiempo de espera adicional se suma al tiempo de transmisión y propagación, generando un incremento en el delay total. Por el contrario, cuando el buffer se vacía o tiene poca ocupación, los paquetes se despachan más rápidamente, y el delay disminuye.

Esto se refleja claramente en las gráficas presentadas: cada pico en la ocupación del buffer coincide con un pico en el delay, y cada fase de alivio corresponde a una caída del retardo.

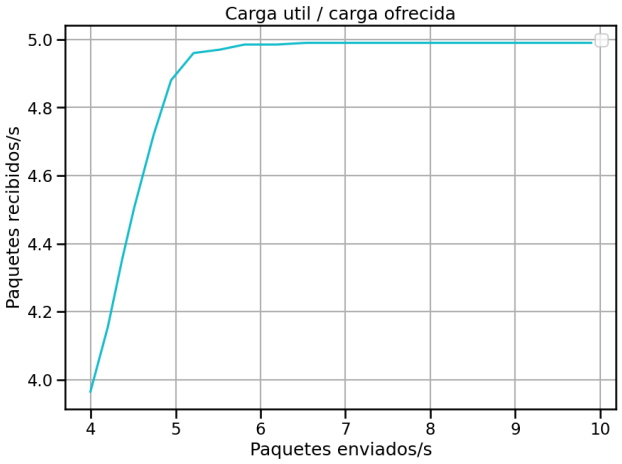


Sin embargo, sigue presente un grave problema de eficiencia que se encontraba también en el control de flujo. Si bien no hay paquetes perdidos, se observa una diferencia entre los paquetes enviados y los recibidos, y esto se debe a que los paquetes faltantes se encuentran aún en el buffer emisor.



Carga útil vs carga ofrecida

El siguiente gráfico muestra la cantidad de paquetes generados por segundo en el eje x, y la cantidad de paquetes recibidos por segundo en el eje y. Se corrieron varias simulaciones (cada punto del gráfico representa una de ellas) donde se modificó el intervalo de generación de paquetes y escenarios de problemas de congestión y de flujo.



Inicialmente, la carga útil crece y se mantiene cerca del valor máximo posible. Esto indica que el sistema es capaz de procesar correctamente casi todos los paquetes generados.

Sin embargo, a medida que la carga ofrecida sigue aumentando, la carga útil deja de crecer y comienza a mantenerse constante, lo que muestra que el sistema ha alcanzado su capacidad máxima de recepción.

A pesar de haberse implementado un sistema de control para flujo y congestión, el gráfico evidencia que existe un límite estructural en la tasa de recepción, más allá del cual los paquetes adicionales no pueden ser procesados eficientemente.

Conclusiones

En ambos casos de estudio, se observaron pérdidas de paquetes por saturación de buffers. En el caso 1, el cuello de botella estaba en el receptor (NodeRx), mientras que en el caso 2, la congestión se generó en la cola intermedia (Queue).

La implementación de control de flujo permitió estabilizar la red al adaptar dinámicamente la velocidad de transmisión según el estado del receptor. Resumidamente funciona de forma que si el buffer del receptor está >50% ocupado, se ralentiza el envío; si está <25% ocupado, se acelera; y si está entre 25% y 50%, se mantiene el ritmo.

Y la implementación del algoritmo de control de congestión se basa en el monitoreo del tiempo de viaje de los paquetes demostró ser eficaz para mantener una ocupación estable de los buffers intermedios. Inicialmente se mide el tiempo de viaje de los primeros 10 paquetes para calcular un promedio de tiempo de envío, luego se analiza el tiempo de viaje de cada nuevo paquete que se reciba, de forma que si un paquete demora más del 20% por encima del promedio, se ralentiza el envío y se resetea el promedio; si demora 80% menos del promedio, se acelera el envío; y si su tiempo de viaje se encuentra entre 20% y 80%, es considerado un tiempo óptimo y no se modifica.

Por otro lado, el algoritmo de control de congestión se implementó de forma que debe utilizarse por separado del de control de flujo, esto debido a que tuvimos problemas para lograr que funcionen bien juntos.

Observamos que los algoritmos por separado funcionan y cumplen con su rol de regular la tasa de paquetes perdidos, brindando una red más confiable mediante el manejo de la velocidad de transferencia de los paquetes. Si bien este método cumple con lo pedido, trae un gran problema que es la acumulación de paquetes en el buffer emisor debido al contexto de la simulación, pues se generan paquetes de manera “infinita” y con muy poco intervalo de generación. Por lo tanto nuestra implementación y el uso de estos umbrales cumplen lo necesario para el estudio de la red planteada en el laboratorio, pero vemos que no es un algoritmo eficiente si el proyecto escalara y tuviera un contexto de simulación parecido.